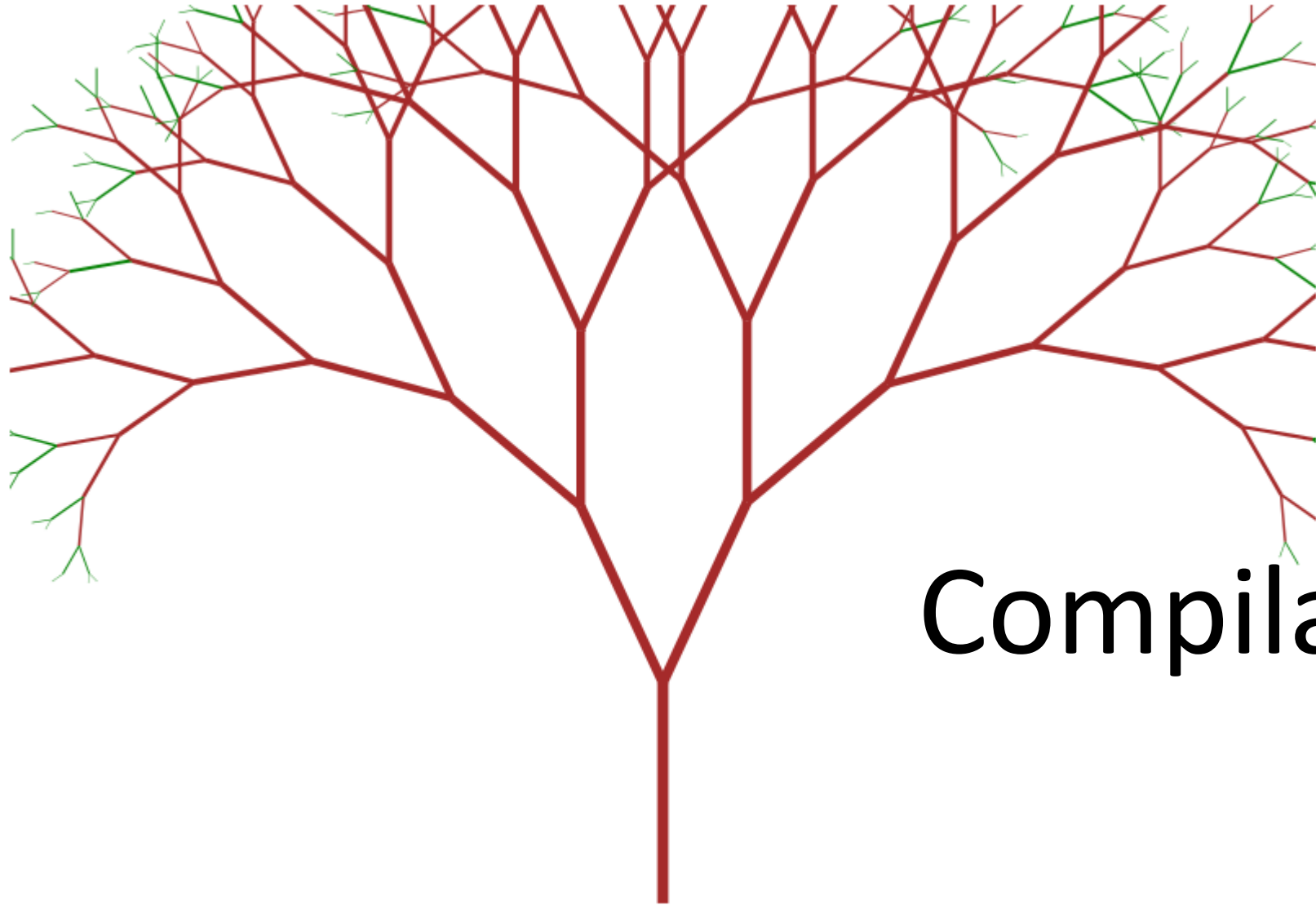




Compiladores

Compiladores

El uso del lenguaje C nos permite describir un cálculo sin referirnos a ninguno de los detalles de la arquitectura ISA, como registros, instrucciones específicas.

```

• #include <stdio.h>

• int main() {

• // Print variables
• int B=5;
• int A=1;

• //printf("Factorial de:");
• //scanf("%d",&B);
• //printf("\n");
• //printf("El Factorial de %d", B);
• // calcula el Factorial
• do {
•     A=A*B;
•     B=B-1;
•     // printf("Resultado: %d\n", Fact);
• }
• while (B!=0);
•     printf("\nResultado : %d", A);
• return 0;
• }
```

Ventajas Lenguajes de alto nivel

- Permiten a los programadores ser muy productivos, ya que los programas son **concisos y legibles**. Estos atributos también facilitan el mantenimiento del código.
- Exactitud del código (Revisión de errores “simples” p.Ej: Guardar un string en variable numérica, comas
- Asignar/liberar memoria
- Portable
- RESULTADO: MENOR TIEMPO EN CREAR UN PROGRAMA EN ALTO NIVEL QUE EN ENSAMBLADOR

Compiladores



También las tareas más complicadas, como la asignación y desasignación dinámica del almacenamiento, pueden automatizarse por completo.



El resultado es que puede llevar mucho menos tiempo crear un programa correcto en un lenguaje de alto nivel que cuando se escribe en lenguaje ensamblador.

Compiladores

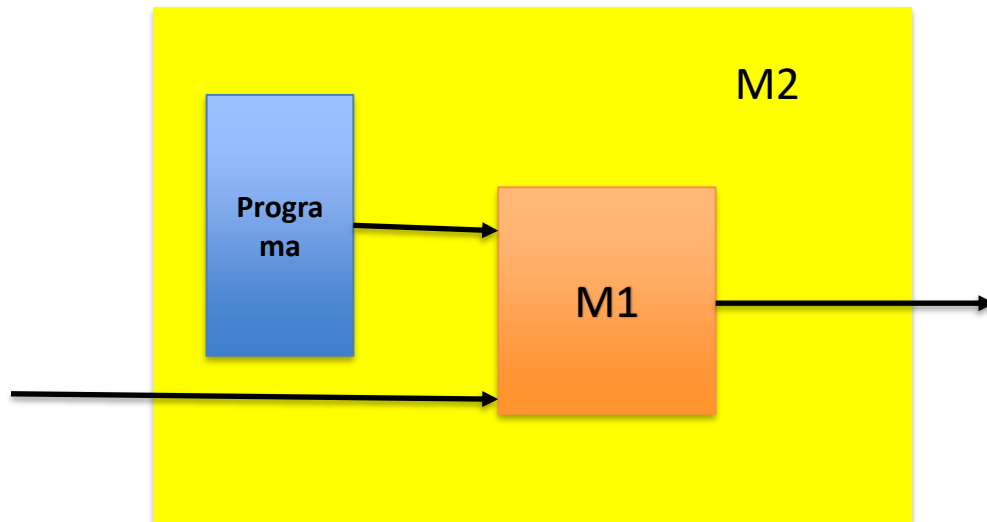
- Dado que el lenguaje de alto nivel ha abstraído los **detalles de un ISA** en particular, los programas son portátiles en el sentido de que podemos esperar ejecutar el mismo código en diferentes ISA sin tener que reescribir el código.



Compiladores

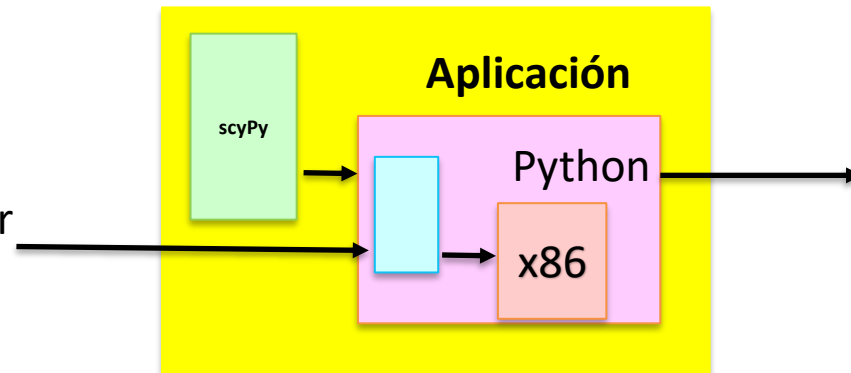
- ¿Qué perdemos al usar un lenguaje de alto nivel?
- Las dos estrategias básicas de ejecución son la **"interpretación"** y la **"compilación"**.

Compiladores: Interpretes



Capas de Interpretación:

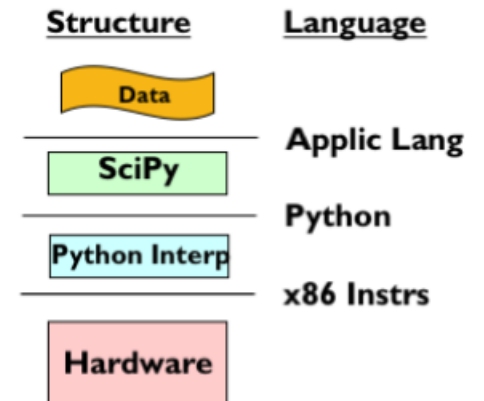
Varias capas de interpretación para obtener el resultado deseado EJ:
X86 CPU, corriendo Python, que corre la aplicación SciPY, que realiza análisis numérico en un programa



Modelo de Interpretación:

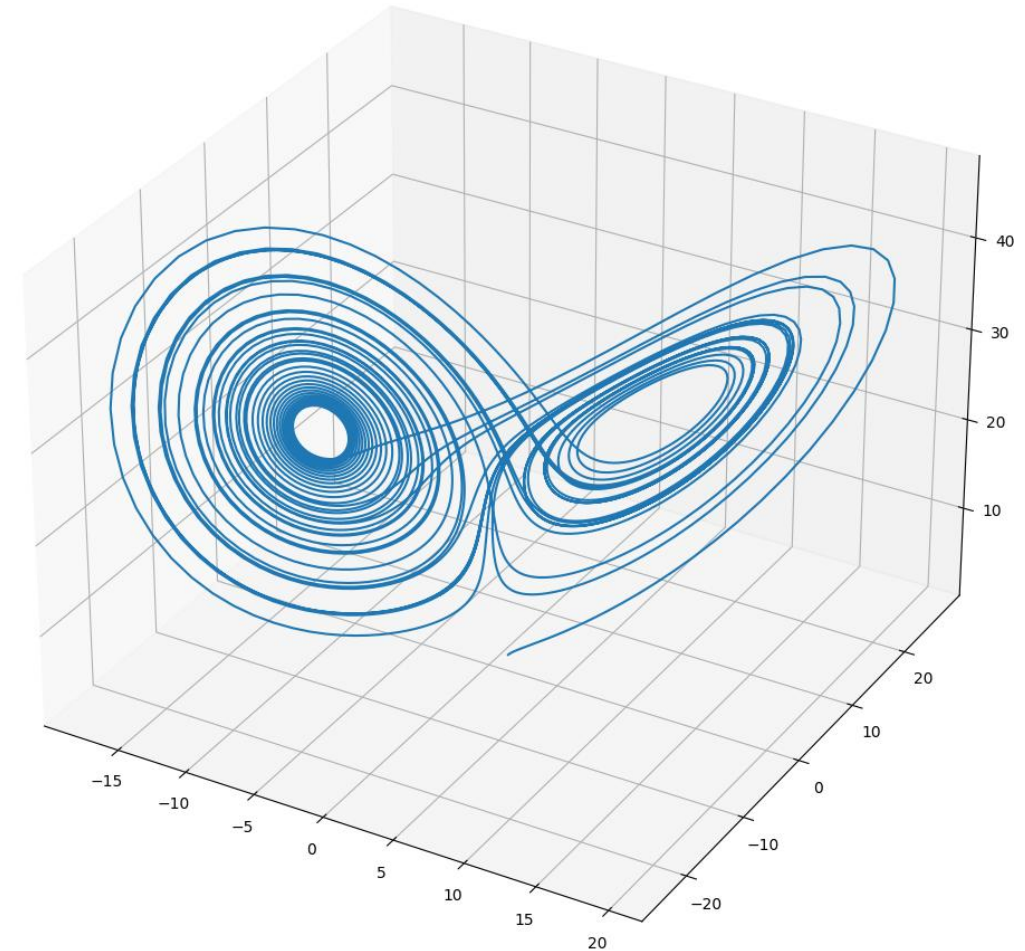
Máquina compleja de Programar: M1
Escribe un programa para M1 que mimitice el comportamiento de una máquina más sencilla: M2

Resultado: Una máquina Virtual M2



Compiladores: Interpretes

- Por ejemplo, un ingeniero puede usar su computadora portátil con una CPU Intel (x86) para ejecutar el **intérprete de Python**. En Python, carga el toolkit de herramientas **SciPy**, que proporciona una interfaz similar a una calculadora para el análisis numérico de matrices de datos.



Compiladores: Interpretes

- Para cada comando de SciPy, por ejemplo, **"encontrar el valor máximo de un conjunto de datos"**, el kit de herramientas de SciPy ejecuta muchas instrucciones de Python, por ejemplo, para recorrer en bucle cada elemento de la matriz, recordando el valor más grande. Para cada instrucción de Python, el intérprete de Python **ejecuta muchas instrucciones x86**,
- Por ejemplo, para incrementar el índice del bucle y comprobar la terminación del bucle.
- La ejecución de un solo comando SciPy puede requerir la ejecución **de decenas de instrucciones de Python**, que a su vez pueden requerir **la ejecución de cientos de instrucciones x86**.
- ¡Muy probablemente ustedes estarían muy contentos de no haber tenido que escribir cada una de esas instrucciones !

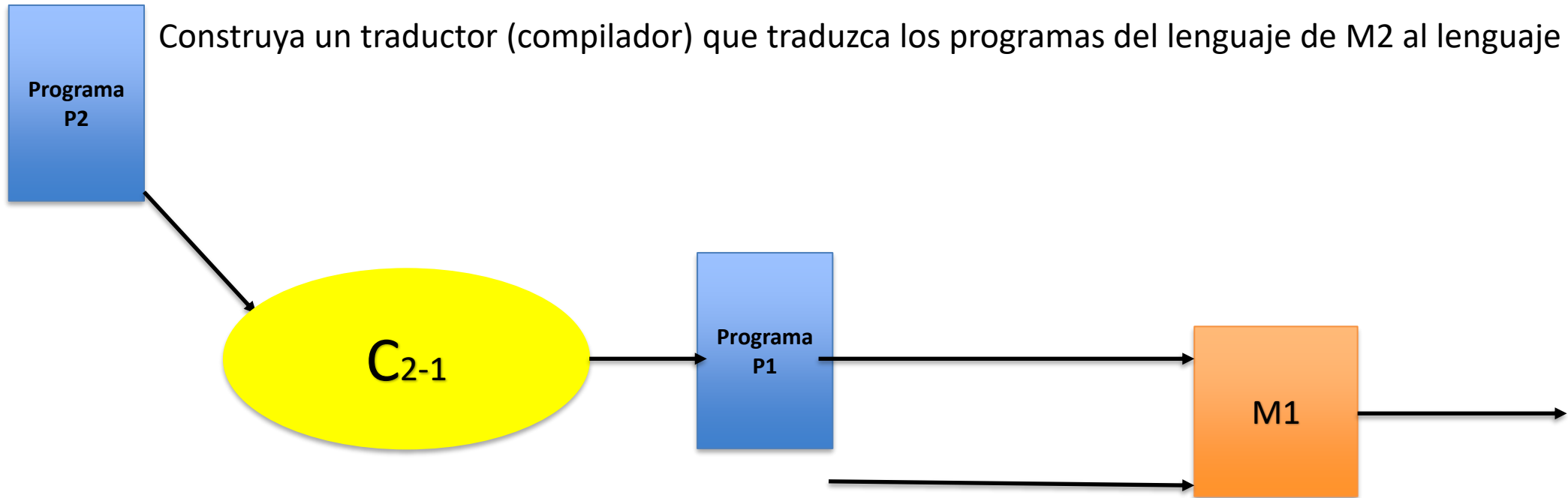
`np.argmax()` GETS THE *INDEX*
OF THE MAXIMUM VALUE OF A NUMPY ARRAY

	1	2	3	100	5
Index:	0	1	2	3	4

Método 2. Compilación

Dado una máquina difícil de programar, (M1), encuentre un lenguaje de programación más sencillo L2, para una máquina (M2) y escriba el programa en ese lenguaje.

Construya un traductor (compilador) que traduzca los programas del lenguaje de M2 al lenguaje de M1.



Compilación

- La etapa de compilación traduce cualquier código fuente a lenguaje ensamblador
- Por ejemplo, gcc produce código en ensamblador para el CPU donde se compila el código; sin embargo, esto se puede cambiar para poder usar el código en otras plataformas.
- En GCC es posible ver el código en ensamblador usando la opción -S.

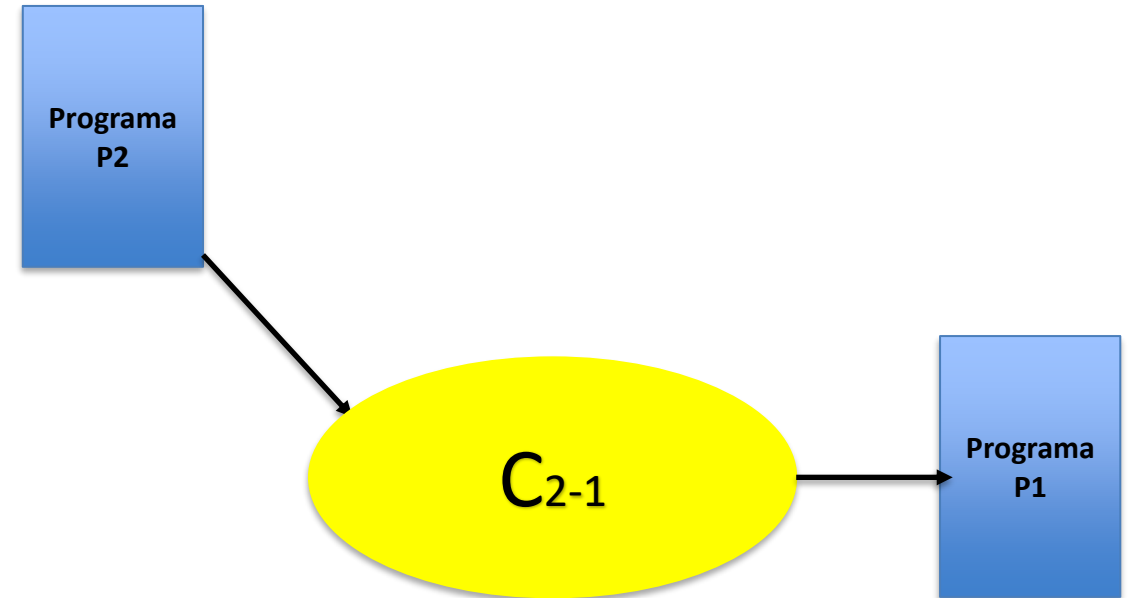
Por ejemplo:

```
gcc -S hello.c
```

```
gcc -c hello.c
```

Compiladores: Compilador

- Utilizamos nuestro programa de lenguaje de alto nivel **P2** y lo traduciremos instrucción por instrucción en un programa para **M1**.
- Tenga en cuenta que en realidad no estamos ejecutando el programa P2.
- En su lugar, lo estamos usando como plantilla para crear un **programa P1** equivalente que pueda ejecutarse directamente en **M1**.
- El proceso de traducción se llama "compilación" y el programa que hace la traducción se llama "compilador".



Compilación

- Si estamos dispuestos a pagar los costos iniciales de la compilación, obtendremos una ejecución más eficiente.
- Con diferentes compiladores, podemos hacer arreglos para ejecutar P2 en muchas máquinas diferentes (M2, M3, etc.) sin tener que reescribir P2.
- Así que ahora tenemos dos formas de ejecutar un programa de lenguaje de alto nivel: interpretación y compilación.
 - Ambos nos permiten cambiar el programa fuente original.
 - Ambos nos permiten abstraer los detalles de la computadora real que usamos para ejecutar el programa.

Compilación vía interprete o vía compilación

Características	Intérprete	Compilación
Cómo se trata la entrada "X + 2"	Computa x+2	Genera un programa que computa x+2
Cuándo sucede	Durante la ejecución	Antes de la ejecución
Qué complica	Ejecución del programa	Desarrollo del programa
Decisiones se toman durante	Run Time	Compile time

- Normalmente la decision más común es si lo hacemos en tiempo de compilación o en RUNTIME.
- Cuál es más rápido
- Cuál es más general

Compiladores comunes

1. **GCC (GNU Compiler Collection):** Un compilador muy popular y versátil que admite múltiples lenguajes de programación, incluidos C, C++ y Fortran.
2. **Clang:** Otro compilador de C/C++, conocido por su rápida compilación y compatibilidad con el framework LLVM (Low Level Virtual Machine).
3. **Microsoft Visual C++:** Un compilador para la plataforma Windows, diseñado específicamente para el desarrollo en C y C++.
4. **Java Compiler (javac):** El compilador oficial de Java utilizado para compilar el código fuente de Java en código de bytes que se ejecuta en la máquina virtual de Java (JVM).

Diferencias entre métodos de compilación

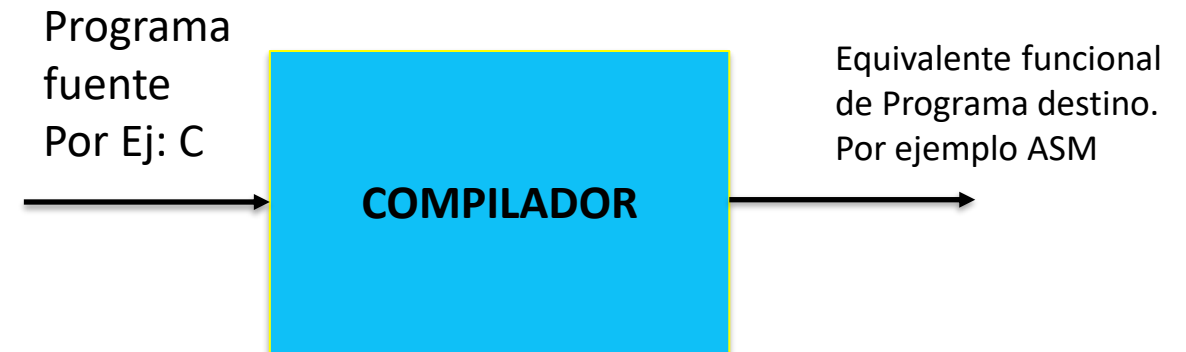
- Vamos a resumir las diferencias entre interpretación y compilación.
- Supongamos que la expresión "**x+2**" aparece en el programa de alto nivel. Cuando el **intérprete procesa** esta expresión, obtiene inmediatamente el valor de la variable x y le agrega 2.
- Por otro lado, el **compilador** generaría **instrucciones** que cargarían la variable x en un registro y le agregaría 2 a ese valor
- Mov AX, X
- Add AX, 02H
- MOV X,AX

Resumen

- El intérprete está ejecutando cada **INSTRUCCIÓN** a medida que se procesa y puede procesar y ejecutar la misma sentencia muchas veces si, por ejemplo, estuviera en un bucle.
- El compilador solo está generando instrucciones para ser ejecutadas en algún momento posterior.
- Los intérpretes tienen el “OVERHEAD” de procesar el código fuente de alto nivel durante la ejecución y ese “overhead” puede ocurrir muchas veces por ejemplo en bucles.
- Los compiladores incurren en el procesamiento del “overhead” una vez, haciendo más eficiente la eventual ejecución. Pero durante el desarrollo, el programador puede tener que compilar y ejecutar el programa muchas veces, a menudo incurriendo en **el costo de compilación** para una sola ejecución del programa.
- Por lo tanto, el ciclo de compilar-correr-debug puede tardar más tiempo.

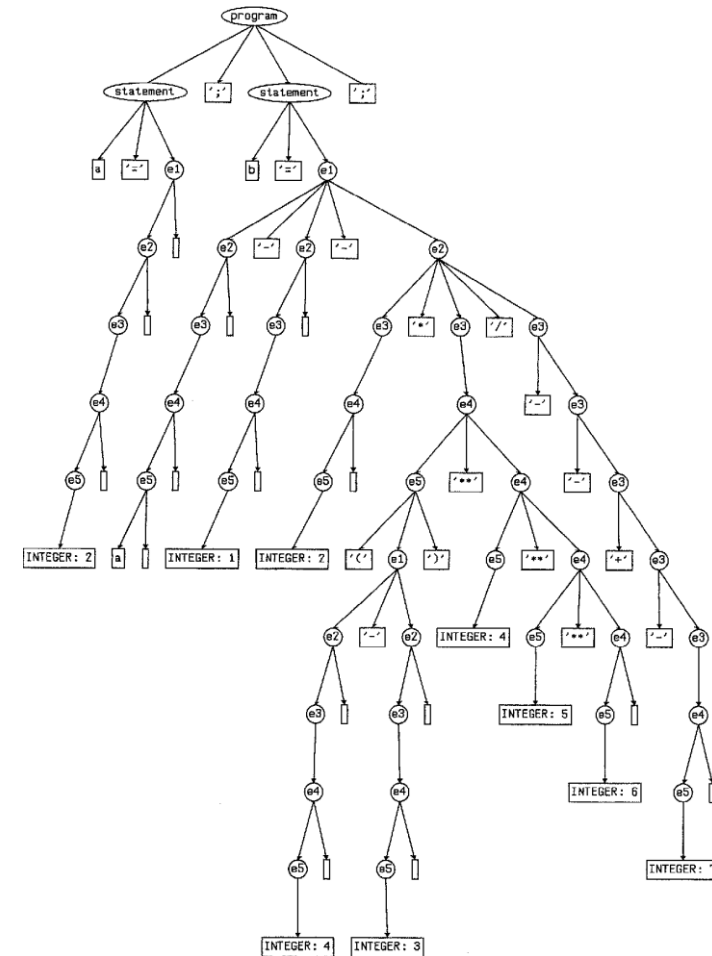
Compilador. Mínimo para ser funcional

- **Buenos compiladores:**
- Producen errores en programas incorrectos. Producen advertencias importantes
- Producen código optimizado y veloz



Estrategia de compilación básica

- Los programas son una serie de declaraciones y expresiones:
- Un compilador necesita al menos un procedimiento llamado por ejemplo:
- call **compila-expresión(expr)**
 - -constantes 1234, variables a, b[expr], asignaciones: a=expr, Operaciones expr1 + expr2, ...
- Y otro llamado por ejemplo **compila Instruccion**
- If, else, while, expresiones, condicionales, etc



Ejemplo de Compilación de expresiones

- **Constante:**

- . Data
 equ 1234h
- Mov AX,1234h

- **Variable**

- . Data
 - VAR DB (?)
- .code
 - MOV VAR,AX

- **Asignación**

- **a=expr;** => Registro
- Compile_expr(expr) => Registro_{CX}
- Mov A, CX

- **Operaciones:**

- expr1+expr2 => Registro
- **Compile_expr(expr1) => Registro1**
- **Compile_expr(expr2) => Registro2**
- Mov AX,expr1
- Mov BX, expr2
- ADD AX,BX

Ejemplo de expresión compilada

Código en c:

```
Int x,y
Y=(x-3)*(y+1234)
```

• Código ensamblado:

- .data
- .code
- **Mov AX,X**
- **Mov bx,3**
- **Sub AX,BX**
- **Mov BX,y**
- **Mov CX,1234**
- **ADD BX,CX**
- **IMUL BX ; AX=AX*BX**
- **MOV Y,AX**

- Compile_expr(Y=(x-3)*(y+1234))
- Compile_expr((x-3)*(y+1234))
- Compile_expr(x-3)
 - **Compile_expr(x)**
 - **Mov AX,x**
- Compile_expr(3)
 - **Mov bx,3**
 - **Sub AX,BX**
- Compile_expr(y+1234)
- Compile_expr(y)
 - **Mov BX,y**
- Compile_expr(1234)
 - **Mov CX,1234**
 - **ADD BX,CX**
 - **IMUL BX ; AX=AX*BX**
 - **MOV Y,AX**

Compilación de declaraciones

C code:

```
if (expr)  
    statement;
```

C code:

```
while (expr)  
    statement;
```

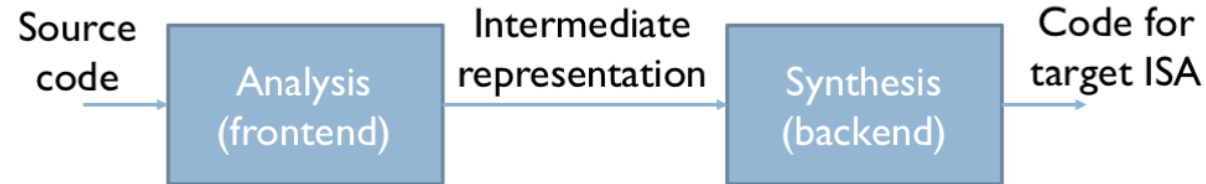
- Por ejemplo:
- Un if es un salto dependiendo si una prueba es falsa o verdadera
- Un while es un “LOOP” o bucle hasta que la prueba se cumpla (Registro de banderas) del CPU.
- Es una generación de código de tareas más pequeñas, la cual genera

Funciones de un compilador

- En general todo compilador moderno realiza
- Análisis del léxico (escaneo)
- Análisis sintáctico (parsing)
- Análisis semántico

Estrategia de compilación

Anatomy of a Modern Compiler

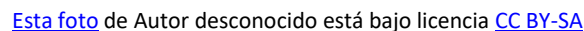


- Read source program
 - Break it up into basic elements
 - Check correctness, report errors
 - Translate to generic **intermediate representation (IR)**
- Optimize IR
 - Translate IR to ASM
 - Optimize ASM

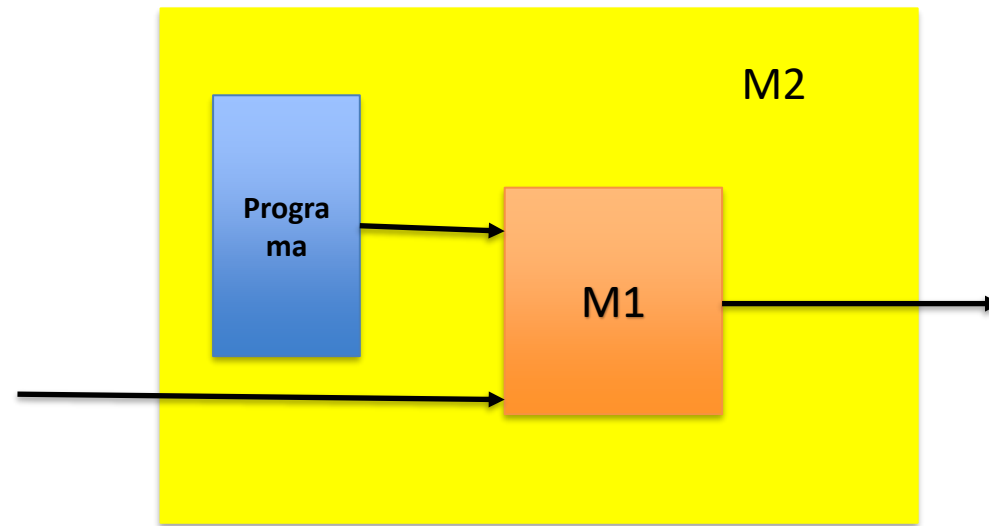
UNA
UNIVERSIDAD
NACIONAL
COSTA RICA

- Se llaman lenguajes de alto nivel porque el programador no tiene que preocuparse por los detalles internos de la CPU.

- Por ejemplo, para escribir un programa en C, uno debe usar un compilador de C para traducir el programa al lenguaje máquina.



Método de Compilación



- La pregunta es, podemos hacer el proceso inverso?